Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ

ДОМАШНЕЕ ЗАДАНИЕ №1

«Реализация алгоритма поиска с опечатками»

ДИСЦИПЛИНА: «Архитектура автоматизированных систем обработки информации и управления»

Выполнил: студент гр. ИУ5-22Б _____ (Леонтьев Д.А.)
(Подпись) (Ф.И.О.)

Проверил: _____ (Гапанюк Ю. Е.)
(Подпись) (Ф.И.О.)

Дата сдачи (защиты):

Результаты сдачи (защиты):

- Балльная оценка:
- Оценка:

Москва, 2026 г.

Условия домашнего задания №1

- Домашнее задание реализуется в виде консольного приложения на языке программирования C#. Рекомендуется использовать операторы верхнего уровня.
- Пользователь вводит две строки, после чего программа выводит значение расстояния Левенштейна между этими строками.
- Указанные выше действия реализуются в виде вечного цикла. Для выхода из цикла в качестве первой строки вводится значение «exit».
- Для сдачи домашнего задания необходимо продемонстрировать работоспособность программы, ответить на вопросы и предоставить отчет, содержащий текст программы и результаты ее выполнения.

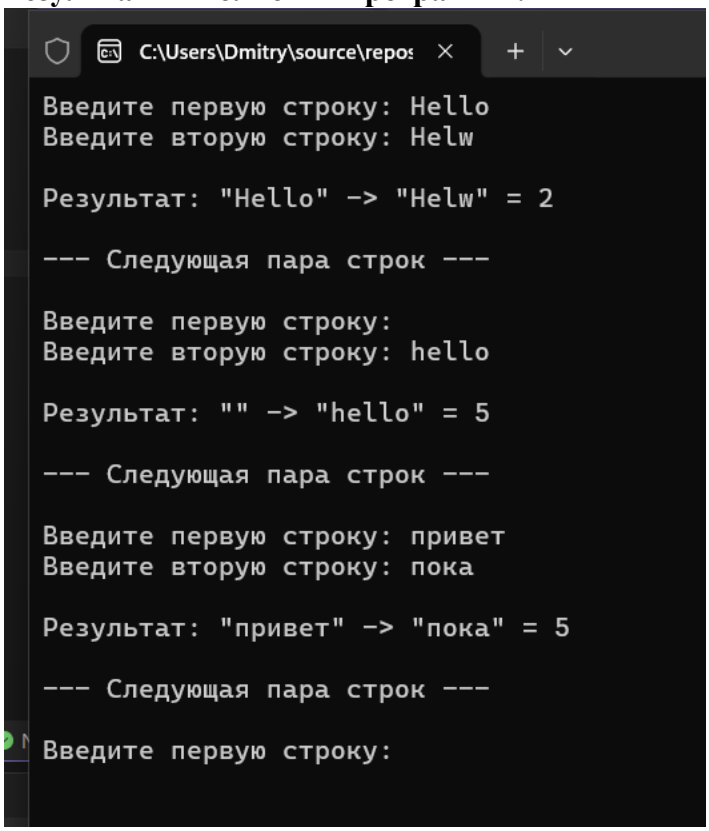
Основной код программы:

```
using System;
using System.Text;
namespace DamerauLevenshteinHomework
{
   class Program
   {
      static void Main(string[] args)
      {
         Console.OutputEncoding = Encoding.UTF8;
         Console.InputEncoding = Encoding.UTF8;
         while (true)
         {
            try
            {
               Console.Write("Введите первую строку: ");
               string? firstInput = Console.ReadLine();

               if (string.Equals(firstInput, "exit", StringComparison.OrdinalIgnoreCase))
               {
                  Console.WriteLine("\nВыход из программы. До свидания!");
                  break;
               }

               if (firstInput == null)
               {
                  Console.WriteLine("Ошибка: ввод не может быть пустым. Попробуйте снова.\n");
                  continue;
               }

               Console.Write("Введите вторую строку: ");
               string? secondInput = Console.ReadLine();

               if (secondInput == null)
               {
                  Console.WriteLine("Ошибка: ввод не может быть пустым. Попробуйте снова.\n");
                  continue;
               }

               int distance = DamerauLevenshteinDistance(firstInput, secondInput);

               Console.WriteLine($"\nРезультат: \"{firstInput}\" -> \"{secondInput}\" = {distance}\n");
               Console.WriteLine("--- Следующая пара строк ---\n");
            }
```

```
        catch (Exception ex)
        {
            Console.WriteLine($"Произошла ошибка: {ex.Message}\n");
        }
    }
}
public static int DamerauLevenshteinDistance(string str1, string str2)
{
    string s1 = str1.ToUpperInvariant();
    string s2 = str2.ToUpperInvariant();

    int n = s1.Length;
    int m = s2.Length;

    if (n == 0) return m;
    if (m == 0) return n;

    int[,] matrix = new int[n + 1, m + 1];

    for (int i = 0; i <= n; i++)
        matrix[i, 0] = i;
    for (int j = 0; j <= m; j++)
        matrix[0, j] = j;

    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= m; j++)
        {
            int cost = (s1[i - 1] == s2[j - 1]) ? 0 : 1;

            int delete = matrix[i - 1, j] + 1;
            int insert = matrix[i, j - 1] + 1;
            int replace = matrix[i - 1, j - 1] + cost;

            matrix[i, j] = Math.Min(Math.Min(delete, insert), replace);

            if (i > 1 && j > 1 &&
                s1[i - 1] == s2[j - 2] &&
                s1[i - 2] == s2[j - 1])
            {
                int transposition = matrix[i - 2, j - 2] + 1;
                matrix[i, j] = Math.Min(matrix[i, j], transposition);
            }
        }
    }

    return matrix[n, m];
}
private static void PrintDistance(string s1, string s2)
{
    int dist = DamerauLevenshteinDistance(s1, s2);
    Console.WriteLine($"'{s1}', '{s2}' -> {dist}");
}
```

```
    }
}
```

Результат выполнения программы:



Ответы на контрольные вопросы:

1. ***Что такое расстояние Левенштейна?***

Расстоянием Левенштейна называется минимальное количество действий, необходимых для преобразования одного слова в другое. Основные операции, учитываемые при вычислении расстояния Левенштейна:

D (delete) — удаление символа

I (insert) — добавление символа

R (replace) — замена символа

M (match) — совпадение символов (не требует действий)

2. ***Чем отличается расстояние Левенштейна от расстояния Дамерау-Левенштейна?***

Расстояние Дамерау-Левенштейна — это усовершенствованная версия расстояния Левенштейна, которая дополнительно учитывает операцию транспозиции (перестановки двух соседних символов) как единичное действие.

При использовании расстояния Дамерау-Левенштейна за единичное расстояние принимаются следующие действия:

I (insert) — добавление символа

D (delete) — удаление символа

R (replace) — замена символа

T (transposition) — перестановка двух соседних символов

3. Объясните алгоритм Вагнера-Фишера

Алгоритм Вагнера-Фишера использует динамическое программирование для вычисления расстояния Левенштейна.

Как работает:

1) Создается матрица размером (M+1) × (N+1), где M и N — длины строк.
2) Заполнение матрицы:
    a. Первая строка: 0,1,2,...,N (вставки)
    b. Первый столбец: 0,1,2,...,M (удаления)
    c. Для каждой ячейки D[i,j] считаем минимум из трех вариантов:
    d. D[i-1, j] + 1 — удаление
    e. D[i, j-1] + 1 — вставка
    f. D[i-1, j-1] + cost — замена (0 если символы совпадают, 1 если разные)

Дополнительно для Дамерау-Левенштейна:

3) Проверяем возможность транспозиции (перестановки соседних символов)
4) Если символы можно переставить, учитываем D[i-2, j-2] + cost
5) Результат — значение в правом нижнем углу матрицы.

Сложность: O(M × N) по времени и памяти.